Game Analytics: Unlocking Tennis Data with Sportradar API

1. Project Overview

This project focuses on building an end-to-end data analytics solution for tennis competition data using the Sportradar API. The system extracts raw JSON data, transforms it into a structured format, stores it in a relational database, and visualizes insights through an interactive web application.

The goal is to help sports analysts, enthusiasts, and organizations explore competition structures, analyze player performance, and derive meaningful insights from tennis event data.

---

2. Objectives

- Extract real-time tennis data using API integration
- Transform nested JSON into structured relational tables
- Design a normalized SQL database
- Perform data analysis using SQL queries
- Build an interactive dashboard for visualization
- Generate actionable insights from the data

---

3. Tech Stack

- **Programming Language:** Python
- **Database:** MySQL / PostgreSQL
- **Visualization & App:** Streamlit
- **Libraries Used:** NumPy, Pandas, requests, sqlalchemy

---

4. System Architecture

Workflow:

API (Sportradar) → Python Scripts → Data Transformation → SQL Database → Streamlit Application → User Insights

---

5. Data Extraction & Transformation

Data Sources:

- Competitions API
- Complexes API

- Doubles Competitor Rankings API

Process:

- API calls made using requests library
- JSON responses parsed and cleaned
- Nested structures flattened into tabular format
- Missing/null values handled appropriately

Key Features:

- Modular Python functions
- Error handling using try-except
- Reusable scripts for scalability

---

6. Database Design

Tables Created:

1. **Categories** – Stores competition categories
2. **Competitions** – Stores competition details
3. **Complexes** – Stores sports complexes
4. **Venues** – Stores venue information
5. **Competitors** – Stores player details
6. **Rankings** – Stores ranking data

Key Design Principles:

- Normalization to avoid redundancy
- Primary and Foreign Keys to maintain relationships
- Efficient schema for querying

---

7. Data Analysis (SQL Queries)

Competition Analysis:

- List competitions with category names
- Count competitions per category
- Identify doubles competitions
- Analyze parent-child competition hierarchy

Venue Analysis:

- List venues with complexes
- Count venues per complex
- Group venues by country
- Identify multi-venue complexes

Ranking Analysis:

- Top 5 ranked competitors
- Stable ranking players
- Country-wise competitor count
- Highest scoring competitors

Additional Insights:

- Distribution of competition types
- Geographic trends of players
- Participation patterns

---

8. Streamlit Application

Features Implemented:

- Interactive dashboard with filters
- Competitor search functionality
- Leaderboards (Top players, highest points)
- Country-wise analysis
- KPI metrics:
    - Total competitors
    - Number of countries
    - Highest points

UI Design:

- Clean and minimal layout
- Interactive widgets (dropdowns, sliders)
- Real-time data fetching from SQL

9. **Key Insights**

- Majority of competitors belong to specific regions, indicating strong tennis presence
- Singles competitions dominate over doubles in most categories
- Certain countries consistently produce top-ranked players
- Competition hierarchy helps understand tournament structure

---

10. Challenges Faced

- Handling deeply nested JSON structures
- Managing API rate limits and errors
- Designing efficient database relationships
- Optimizing SQL queries for performance
- Building a responsive and user-friendly UI

---

11. Solutions Implemented

- Flattened JSON using structured parsing techniques
- Implemented error handling and retry mechanisms
- Used normalization and indexing in database
- Optimized queries using joins and grouping
- Applied clean UI design principles in Streamlit

---

12. Project Evaluation Metrics

- Accurate data extraction from API
- Well-structured and normalized database
- Efficient and meaningful SQL queries
- Functional and interactive application
- Clear documentation and usability

---

13. Conclusion

This project demonstrates an end-to-end data analytics workflow, from data extraction to visualization. It highlights practical skills in API integration, database management, SQL analysis, and dashboard development. The solution provides a scalable and interactive platform for exploring tennis data, making it valuable for both analytical and business decision-making purposes.

14. How to Run the Project

1. Clone the repository
2. Install required libraries:
3. pip install -r requirements.txt
4. Set up database and import schema
5. Run API scripts to populate data
6. Launch Streamlit app:
7. streamlit run app.py

---

15. Repository Structure

```
project/
|
 ├── data/
 ├── scripts/
|   ├── CompitititionsAPI.py
|   ├── ComplexesAPI.py
|   ├── rankingsAPI.py
|
 ├── sql/
|   └── sql_queries.sql
|
 ├── app/
|   └── app.py
|
 ├── docs/
|   └── Documentation.pdf
|
└── README.md
```

---

16. Author

- Shreya Thaware – DA/BA Batch 1st Oct
- Rupali Jain - DA/BA Batch 15th Oct
- Muskan Bhardwaj - DA/BA Batch 15th Oct
- Guransh Dhiman - DA/ BA Batch 15th Oct

---

17. Acknowledgment

This project was completed as part of an internship program, focusing on real-world data analytics applications using modern tools and technologies.